

Data Structures — Condensed Exam Prep (6000 Words)

ASE Bucharest · Prof. Mihai Doinea · 339 MCQ Coverage

Ch 1 — Pointers & Memory Model

Definition: A pointer is a variable storing a memory address. Declared with `*(int* p)`, dereferenced with `*p`.

Memory regions:

- **Stack:** local vars, auto-freed when function returns, limited size (~few MB)
- **Heap:** malloc/free, manual management, much larger
- `malloc(n)` returns `void*` to `n` bytes on heap; `free(p)` returns those bytes but **does not change** `p`

Key operations: Declare $O(1)$, dereference $O(1)$, malloc $O(1)$, free $O(1)$

Canonical code trace:

```
int* pc, c, d;
pc = (int*)malloc(sizeof(int));
pc[0] = -10;
c = *pc + 11;    // c = -10 + 11 = 1
d = c + 1;       // d = 2
printf("c=%d, d=%d, *pc=%d", c, d, *pc); // c=1, d=2, *pc=-10
free(pc);        // p still holds the address
```

Exam traps:

- `free(p)` does NOT set `p` to NULL — you must do it yourself
- `int* p; *p = 10;` — uninitialized pointer, undefined behavior
- `char arr[10]` last valid index = 9 (not 10)
- Array name decays to pointer: `int* pv = v;` then `v[0] == pv[0]`
- Static arrays on stack (no free needed), dynamic arrays on heap (must free)

Ch 2 — Bits, Memory Classes & Unions

Storage classes:

Specifier	Storage	Init	Scope	Lifetime
auto (default)	Stack	Garbage	Block	Block exit
static	Data	Zero	Block	Program
extern	Data	Zero	Global	Program
register	CPU reg	Garbage	Block	Block exit

Bitwise operators: & AND, | OR, ^ XOR, ~ NOT, << left shift, >> right shift. All $O(1)$.

Bit fields: `struct { unsigned int is_valid : 1; int : 3; };` — access bits by name, can't take address.

Union: All members share same memory. Size = largest member. Only one active at a time.

Exam traps:

- `extern` declares but doesn't define (no memory allocated)
 - Signed right-shift is implementation-defined
 - Can't take address of bit field
 - `typedef` vs `#define`: `typedef int* P; P a,b;` makes both pointers; `#define P int*` makes only a a pointer
-

Ch 3 — Arrays

Definition: Contiguous memory block. `int arr[10]` — 10 ints on stack. `int* arr = malloc(10*sizeof(int))` — 10 ints on heap.

Key facts:

- Access $O(1)$: `arr[i] = *(arr + i) = base + i*sizeof(type)`
- Search $O(n)$ unsorted, $O(\log n)$ sorted (binary search)
- Insert/delete $O(n)$ (shift required)
- Last valid index of `arr[n]` is `n-1`

Dynamic array (vector): Start with capacity C , when full double to $2C$. Amortized $O(1)$ insert at tail. $O(n)$ insert at middle.

Exam traps:

- `sizeof(arr)` vs `sizeof(ptr)` — array gives total bytes, pointer gives 4/8
 - Array bounds not checked in C — buffer overflow is silent
 - `realloc` may move memory — invalidate all pointers
-

Ch 4 — Linked Lists

Definition: Chain of heap-allocated nodes. Each node: data + pointer(s) to next/prev.

Variants:

- **Singly:** next only. Last $\rightarrow$ NULL
- **Doubly:** next + prev. $O(1)$ deletion given node pointer
- **Circular:** last $\rightarrow$ first (no NULL)
- **Circular Doubly:** last $\leftrightarrow$ first

Operations:

Operation	Singly	Doubly
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$ or $O(1)$ with tail ptr	$O(1)$ with tail ptr
Delete given node	$O(n)$ to find prev	$O(1)$
Search	$O(n)$	$O(n)$

Exam traps:

- Passing head by value doesn't update caller's head (need `Node**` or return head)
- List nodes at random heap addresses (not contiguous)

- Circular list: last node points to FIRST, not NULL
- DLL is best for $O(1)$ arbitrary deletion (have both neighbors)

Ch 5 — Stacks & Queues

Stack (LIFO): push, pop, peek — all $O(1)$. Array-based: top index. List-based: head is top.

Queue (FIFO): enqueue, dequeue, front — all $O(1)$. Circular buffer: head/tail with mod arithmetic. List-based: head for dequeue, tail for enqueue.

Use cases: Stack → function calls, undo, DFS, expression eval. Queue → BFS, task scheduling, buffering.

Exam trap: "Stack" (data structure) ≠ "stack" (memory region). Heap data structure ≠ heap memory region.

Ch 6 — Hash Tables

Definition: Array of buckets. Hash function maps key → index. Collision resolution: chaining (linked list per bucket) or open addressing (probe next slot).

Operations: Insert $O(1)$ avg, $O(n)$ worst. Search $O(1)$ avg. Delete $O(1)$ avg.

Load factor $\alpha = n/m$ (items/slots). When $\alpha > \text{threshold}$ (e.g., 0.75), resize and rehash — $O(n)$.

Chaining: Each bucket = linked list. Insert at head of chain. Search walks chain.

Linear probing: On collision, check next slot, then next, etc. Cluster formation degrades performance.

Exam traps:

- Hash tables don't preserve ordering
- Worst case $O(n)$ — all keys hash to same bucket
- Load factor determines performance, not table size
- Rehashing is expensive but amortized $O(1)$

Ch 7 — Binary Heaps

Definition: Complete binary tree with heap property. Min-heap: parent $\leq$ children. Max-heap: parent $\geq$ children.

Array storage (0-indexed): parent = $(i-1)/2$, left = $2i+1$, right = $2i+2$

Operations:

- Insert: append + bubble up → $O(\log n)$
- Extract min/max: swap root with last, remove last, bubble down → $O(\log n)$
- Peek: $O(1)$
- Build heap (Floyd's): $O(n)$ — not $O(n \log n)$

Exam traps:

- Heap ≠ sorted array (only partially ordered)
- Heap ≠ BST (no left < parent < right invariant)
- Build-heap is $O(n)$, not $O(n \log n)$
- 0-indexed: left= $2i+1$, right= $2i+2$. 1-indexed: left= $2i$, right= $2i+1$

Ch 8 — Maps & Sets

Map: Key→value pairs. Keys unique. Implementations: HashMap ($O(1)$ avg) or TreeMap ($O(\log n)$ ordered).

Set: Unique elements only. Union $O(n+m)$, intersection $O(\min(n,m))$ for hash-based.

C has no built-in map/set — implement via hash table or BST.

Exam trap: Map keys must be unique; inserting duplicate key overwrites value.

Ch 9 — Binary Search Trees

Definition: Binary tree where left subtree < node < right subtree. In-order traversal yields sorted sequence.

Operations:

- Insert: walk down, attach as leaf → $O(\log n)$ avg, $O(n)$ worst
- Search: walk left/right based on comparison → $O(\log n)$ avg
- Delete: 3 cases — leaf (remove), 1 child (replace with child), 2 children (replace with in-order successor/predecessor)

Traversals: In-order (L-Root-R, sorted for BST), Pre-order (Root-L-R), Post-order (L-R-Root), Level-order (BFS with queue).

Exam traps:

- BST gives sorted via in-order ONLY
 - Degenerate BST = linked list = $O(n)$ operations
 - Delete with 2 children: use successor (min of right subtree) or predecessor (max of left subtree)
-

Ch 10 — AVL Trees

Definition: Self-balancing BST. Balance factor = $\text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, +1\}$ for every node.

4 rotation cases:

- **RR (right-right):** single left rotation
- **LL (left-left):** single right rotation
- **LR (left-right):** left rotate left child, then right rotate root
- **RL (right-left):** right rotate right child, then left rotate root

Operations: Insert $O(\log n)$ with at most 2 rotations. Delete $O(\log n)$ with up to $O(\log n)$ rotations. Search $O(\log n)$.

Height: AVL with n nodes has height $\leq 1.44 \log_2(n)$. Guarantees $O(\log n)$ worst case.

Exam traps:

- Balance factor direction: $BF = \text{height}(\text{left}) - \text{height}(\text{right})$, NOT right - left
 - Insert needs 0-2 rotations; delete can need $O(\log n)$
 - AVL is not the only balanced tree (Red-Black, B-tree, etc.)
-

Ch 11 — B-Trees

Definition: Multi-way search tree of order m . Each node: up to m children, up to $m-1$ keys. All leaves at same height.

Properties: Height = $O(\log_m n)$. One node $\approx$ one disk page. Used in databases/filesystems.

B+ tree variant: All data in leaves only. Internal nodes hold routing keys. Leaves linked for sequential access.

Operations: Search/insert/delete $O(\log_m n)$. Insert: split node when full. Delete: merge or redistribute.

Exam traps:

- B-tree $\neq$ binary tree (multi-way)
- All leaves at same height (perfectly balanced)
- Node splitting is bottom-up

Ch 12 — Graphs

Definition: $G = (V, E)$. Directed/undirected, weighted/unweighted.

Representations:

- Adjacency matrix: $O(V^2)$ space, $O(1)$ edge lookup
- Adjacency list: $O(V+E)$ space, $O(\text{degree})$ edge lookup

Traversals:

- BFS: queue, level-by-level, $O(V+E)$, finds shortest path in unweighted graph
- DFS: stack/recursion, depth-first, $O(V+E)$, detects cycles, topological sort

Topological sort: Order vertices so all edges go forward. Only works on DAGs (no cycles).

Exam traps:

- BFS uses queue, DFS uses stack/recursion
- Undirected graph: adjacency matrix is symmetric
- Graph $\neq$ tree (tree is acyclic connected graph)

Ch 13 — Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes

Lower bound: $\Omega(n \log n)$ for comparison-based sorts.

Exam traps:

- Quick sort worst case $O(n^2)$ — bad pivot choice
- Merge sort stable but $O(n)$ space
- Insertion sort best for nearly-sorted data ($O(n)$)
- Heap sort in-place $O(1)$ space but not stable

Ch 14 — Applications & Library Functions

String ops: `strlen` $O(n)$, `strcpy` $O(n)$, `strcmp` $O(n)$, `strcat` $O(n)$. All null-terminated.

Base conversion: Decimal to binary — repeatedly divide by 2, collect remainders (stack needed for reversal).

Infix to postfix: Dijkstra Shunting-Yard algorithm — operator precedence stack.

Compression: Run-length encoding (AAABBB $\rightarrow$ 3A3B). Huffman coding: variable-length codes via frequency-sorted binary tree.

C library: `malloc/free/calloc/realloc` (`stdlib.h`), `strlen/strcpy/strcmp/strcat` (`string.h`), `qsort/bsearch` (`stdlib.h`).

Quick Reference — All Big-O

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Dynamic Array	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$ head	$O(1)$ given node
Hash Table	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
BST	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	—	$O(n)$	$O(\log n)$	$O(\log n)$
B-Tree	$O(\log_m n)$	$O(\log_m n)$	$O(\log_m n)$	$O(\log_m n)$
Graph BFS/DFS	—	$O(V+E)$	—	—

*amortized for dynamic array tail insert

Top 10 Exam Traps

1. `free(p)` does NOT set `p` to NULL — dangling pointer
2. Static arrays on stack, dynamic on heap — don't `free` stack memory
3. Last index of `arr[n]` is `n-1`, not `n`
4. `sizeof(arr) != sizeof(ptr)` when `arr` decays to pointer
5. Heap data structure $\neq$ heap memory region
6. BST inorder = sorted. AVL guarantees $O(\log n)$. Degenerate BST = $O(n)$

7. Quick sort $O(n^2)$ worst case. Merge sort $O(n)$ space.
8. Hash table worst case $O(n)$ — don't assume $O(1)$ always
9. Circular list: last $\rightarrow$ first, NOT last $\rightarrow$ NULL
10. Passing head by value doesn't update caller's head — use `Node**`